# Smart Cloud Shield (SCS)

System Architecture & Technical Viva Reference Guide

## 1. System Overview & Problem Statement

**Core Objective:** Traditional cloud storage models rely solely on standard perimeter access controls. If credentials leak or an insider accesses the physical server storage, stored data is entirely exposed.
**Smart Cloud Shield (SCS)** implements an end-to-end **Defense-in-Depth** security model combining three pillars: **Multi-Factor Authentication (MFA)** at entry, **Server-Side Encryption at Rest** on storage, and **Real-Time Behavioral Anomaly Detection** to detect and isolate threats dynamically.

## 2. Architectural Components & Implementation

| Component | Implementation Details | Key Security Purpose |
|---|---|---|
| **Authentication & MFA** | Password hashed with salted algorithm + time-limited 6-digit dynamic OTP dispatched via Brevo REST API over HTTPS port 443. | Protects against credential theft, automated bots, and brute-force logins. |
| **Data Encryption (At Rest)** | Cryptographic symmetric encryption (Fernet / AES). Files are encrypted before disk write; decrypted only in-memory upon download. | Ensures that direct server storage breaches yield only unintelligible ciphertext. |
| **AI Threat & Risk Engine** | Analyzes user telemetry (failed attempt count, unrecognized IP addresses, device user-agents, request velocity) into a 0–100 score. | Flags anomalous behavior early and automatically logs security incidents for admins. |
| **RBAC & Audit Pipeline** | Two role tiers: Standard Users (file management) and Administrators (system monitoring, user management, incident logs). | Maintains strict principle of least privilege and immutable audit trails for forensic tracking. |

## 3. End-to-End Operational Workflows

• **Authentication Lifecycle:** User inputs username & password → Credentials verified → Server generates cryptographically secure 6-digit OTP (10 min TTL) → Dispatched via Brevo HTTPS API → User enters OTP → Session created with role-based JWT/cookie.
• **Secure File Upload Lifecycle:** User submits file multipart payload → Metadata extracted & stored in DB → Raw payload encrypted with symmetric cipher → Ciphertext written to storage volume → Audit log generated.
• **Threat Incident Lifecycle:** Anomaly detected (e.g. 5 failed logins from new IP) → Telemetry engine calculates risk index > 70 → Incident record generated → Admin alert dispatched & visible on live dashboard.

## 4. Technical Viva & Presentation Questions with Model Answers

### Q1: What is the core problem Smart Cloud Shield solves?

**Answer:** Traditional cloud drives rely solely on perimeter authentication. If a password leaks or the storage volume is accessed directly, files are compromised. SCS enforces Defense-in-Depth: mandatory email MFA, server-side encryption before writing to disk, and real-time behavioral anomaly scoring.

### Q2: How does the encryption at rest work in this system?

**Answer:** When a user uploads a file, the server encrypts the raw byte stream using AES/Fernet symmetric encryption before storing it on disk. Even if an attacker compromises the server filesystem, they only see ciphertext. Decryption happens strictly in-memory when an authenticated user requests a download.

### Q3: Why use Brevo's HTTPS REST API instead of standard SMTP port 587?

**Answer:** Free and cloud container platforms (like Render Free, AWS sandboxes, and restricted college networks) deliberately block outbound traffic on standard SMTP ports (25, 465, 587) to prevent spam abuse. Using an external transactional provider's REST API over HTTPS (port 443) bypasses port blocks and guarantees reliable OTP delivery.

### Q4: How does the Risk Scoring engine calculate threat levels?

**Answer:** The engine evaluates contextual indicators: login failure velocity, unrecognized IP addresses, device user-agent changes, and abnormal request patterns. It computes a normalized score (0–100). Scores exceeding established thresholds trigger security alerts and automated incident logging on the Admin dashboard.

### Q5: What is the full technology stack and why was each chosen?

**Answer:** Backend: Python & Flask for lightweight, modular microservices. Database: SQLite with SQLAlchemy ORM for relational modeling of users, file metadata, logs, and incidents. Cryptography: Python's cryptography library for Fernet/AES symmetric ciphers. Frontend: Semantic HTML5, CSS custom design with dark-mode telemetry dashboards. Email: Brevo transactional HTTPS API.

### Q6: How is Role-Based Access Control (RBAC) enforced?

**Answer:** Users belong to either 'User' or 'Admin' role. Standard users only have access to their isolated files and profile endpoints. Administrators have elevated privileges to inspect system-wide audit trails, view risk telemetry gauges, manage user accounts, and resolve security incidents.

### Q7: What happens during a brute-force attack on a user's account?

**Answer:** Repeated authentication failures immediately increase the telemetry risk index, trigger an audit event, and create a high-priority incident on the Admin dashboard. Even if the password is breached, authentication cannot complete without the dynamic, time-limited 6-digit OTP delivered to the verified email address.

### Q8: What is stored in the database versus what is stored on the disk filesystem?

**Answer:** The database stores structured metadata: user credentials (hashed), file metadata (filename, owner ID, file hash, size, timestamp), audit logs, and security incidents. The physical disk volume only stores encrypted ciphertext blocks, ensuring separation of metadata from data payload.

---